

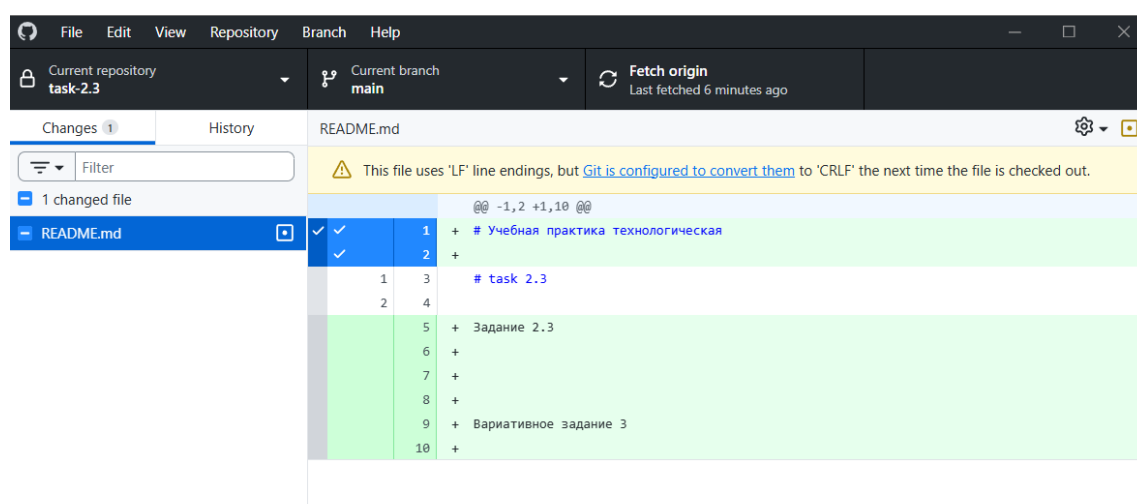
Цель работы: изучить интерфейс, внутренние защитные алгоритмы и степень автоматизации наиболее распространенных команд в графическом клиенте GitHub Desktop при выполнении стандартных и продвинутых операций контроля версий.

В GitHub Desktop был создан приватный репозиторий task-2.3.

С целью внесения изменений в отслеживаемый документ, файл README.md был открыт в текстовом редакторе «Блокнот». На начальном этапе файл содержал единственную тестовую строку.

В структуру файла были внесены дополнительные строки, после чего файл был сохранен. При возвращении в интерфейс GitHub Desktop встроенный демон мониторинга файловой системы автоматически зафиксировал дельту изменений на вкладке Changes. Графический парсер клиента деконструировал файл и представил новые строки с цветовой индикацией.

Вместо вызова интерактивной консольной команды интерактивного индексирования `git add`, требующей ручного разбора патчей в терминале, инструмент предоставил графические чекбоксы слева от номеров строк. Были выбраны только первые две строки модифицированного текста. Остальная часть добавленного кода осталась в области неотслеживаемых изменений рабочей директории.

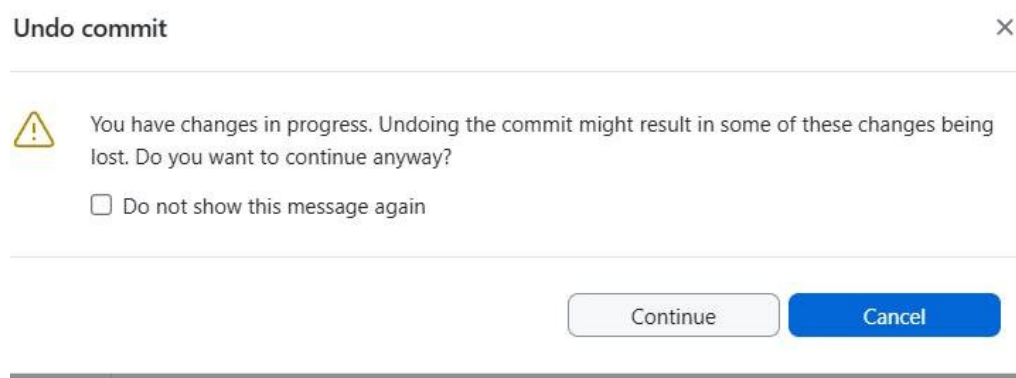


Выборочное индексирование строк 1 и 2 с помощью синих графических чекбоксов в GitHub Desktop.

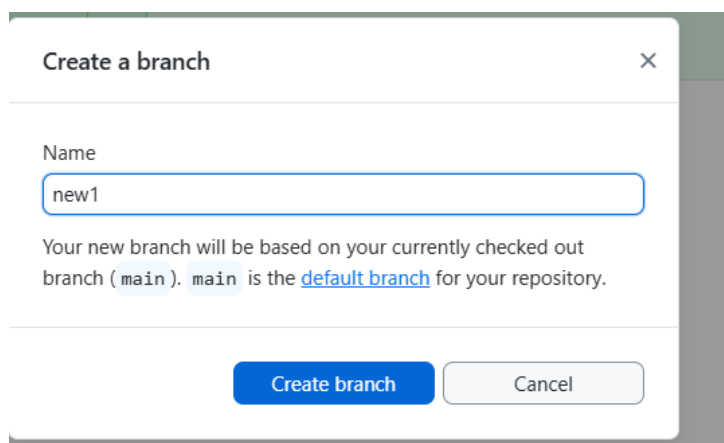
После заполнения заголовка коммита была выполнена локальная фиксация состояния ветки. При переходе на вкладку History отобразился созданный коммит, содержащий только выбранный ранее фрагмент кода. Реализован частичный КОММИТ.

Далее было проведено тестирование защитных механизмов графического интерфейса при попытке инициировать откат индекса последнего коммита с помощью функции Undo в условиях, когда в рабочей директории присутствовали незафиксированные изменения.

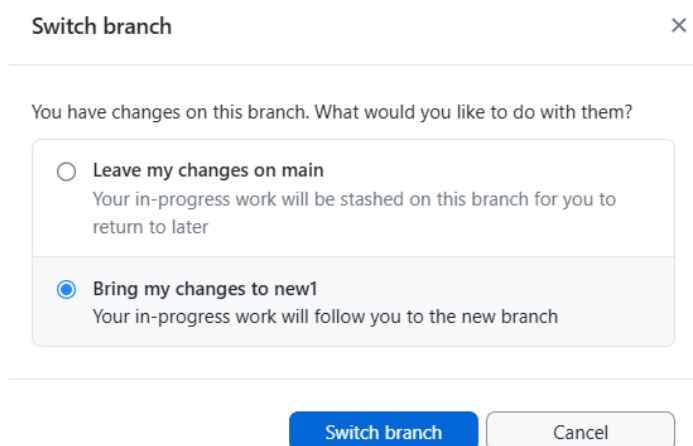
Алгоритм GUI-клиента заблокировал мгновенное выполнение операции, распознав угрозу коллизии данных, и вывел транзитное предупреждение «Undo commit». Программа программно защищает текущий прогресс от непреднамеренной перезаписи, требующей ручного восстановления. Для принудительного выполнения перезаписи указателя локальной ветки была нажата кнопка Continue.



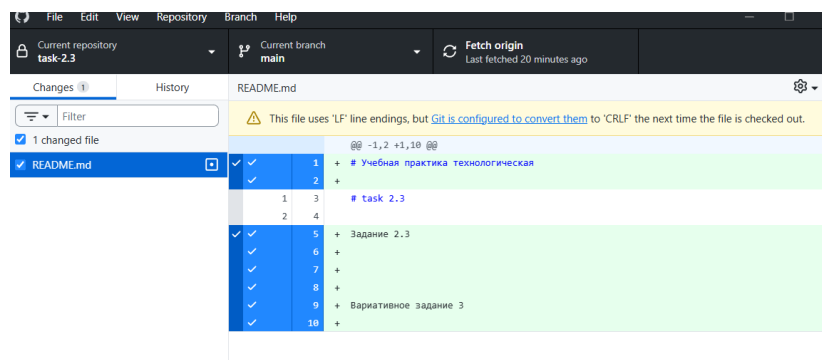
В меню выбрана опция New Branch. В диалоговом окне «Create a branch» было задано имя новой ветки - new1.



Перед переключением в ветке main остались незафиксированные строки. При попытке сменить ветку программа открыла диалоговое окно «Switch branch» со сценарием переноса файлов в новую ветку. Выбрана опция «Bring my changes to new1» и нажал Switch branch.



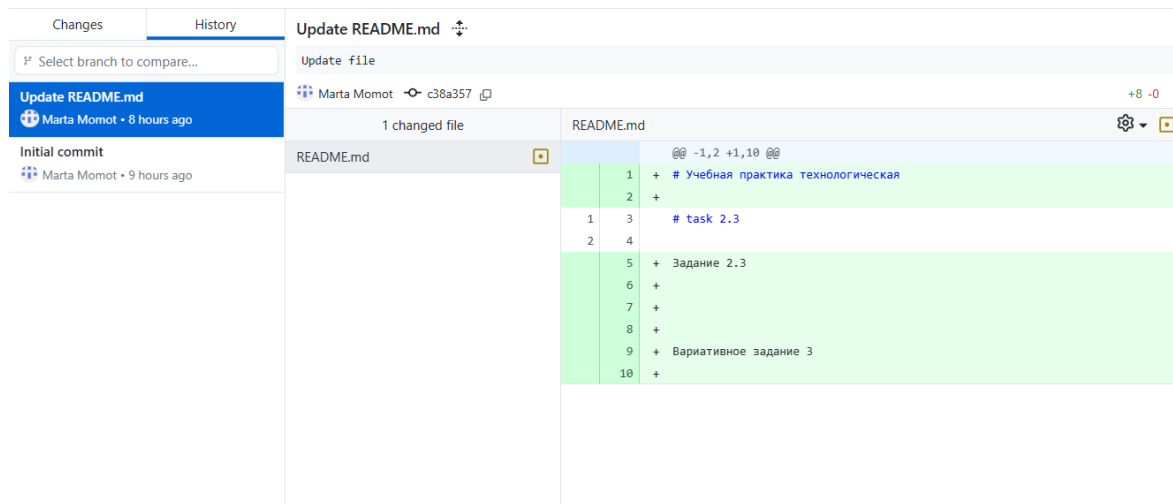
Изменения перенесли в ветку new1. Во вкладке Changes выбран файл README.md.



Программа показала все добавленные строки. Затем сделан коммит кнопкой Commit 1 file to new1.

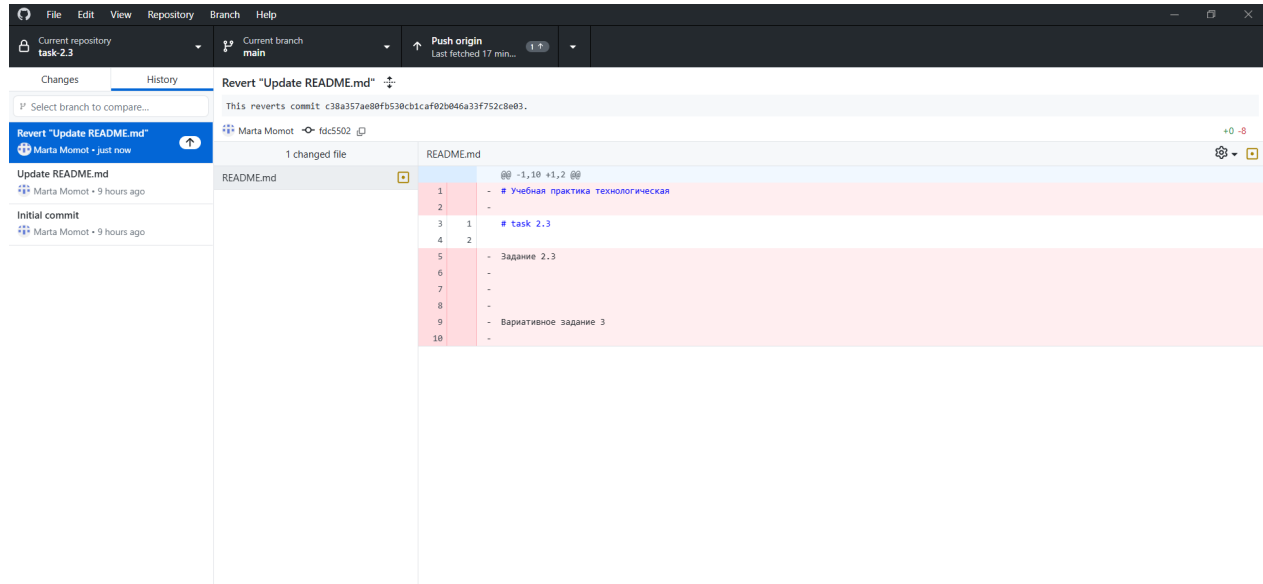
После успешной фиксации кода в ветке new1 была выполнена интеграция изменений в основную ветку. Для этого была переключена ветка на main. Затем выбрана опция Choose a branch to merge into main и указана ветку new1. Программа провела операцию слияния веток.

Интерфейс продемонстрировал готовность всех модифицированных строк файла к фиксации. Выполнена запись изменений в историю локальной ветки.



Далее исследовался механизм неразрушающего отката изменений. Вместо использования команды `git reset --hard`, в интерактивной ленте истории была вызвана функция `Revert changes in commit`.

Программа автоматически создала коммит с заголовком `Revert «Update README.md»`. Все добавленные строки были удалены, это подсвечено красным цветом в области diff. Целостность и линейность хронологии репозитория не были нарушены.

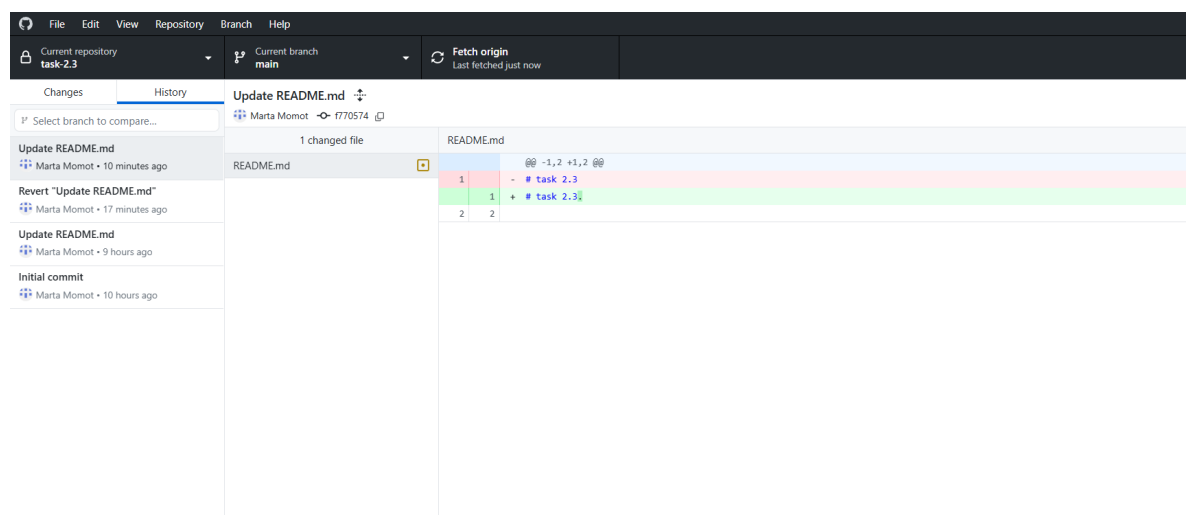


Синхронизация состояния репозитория

Для проверки взаимодействия с удаленным сервером через веб-интерфейс `github.com` в репозиторий были внесены внешние изменения, в результате чего указатель удаленной ветки на сервере опередил локальный HEAD компьютера. При возвращении в клиент и автоматическом или ручном выполнении команды

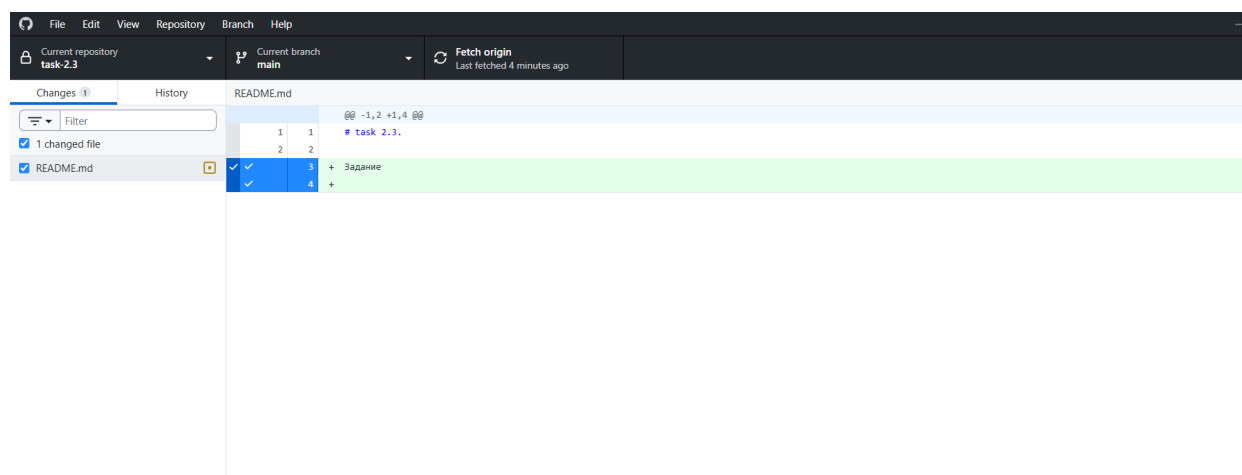
Fetch origin программа идентифицировала расхождение версий и перестроила интерфейс кнопки синхронизации, отобразив статус Pull origin с указанием количества входящих коммитов.

Локальная рабочая копия была синхронизирована посредством нажатия кнопки Pull origin. Клиент успешно выполнил слияние перемоткой вперед, отобразив полученный коммит в истории изменений.



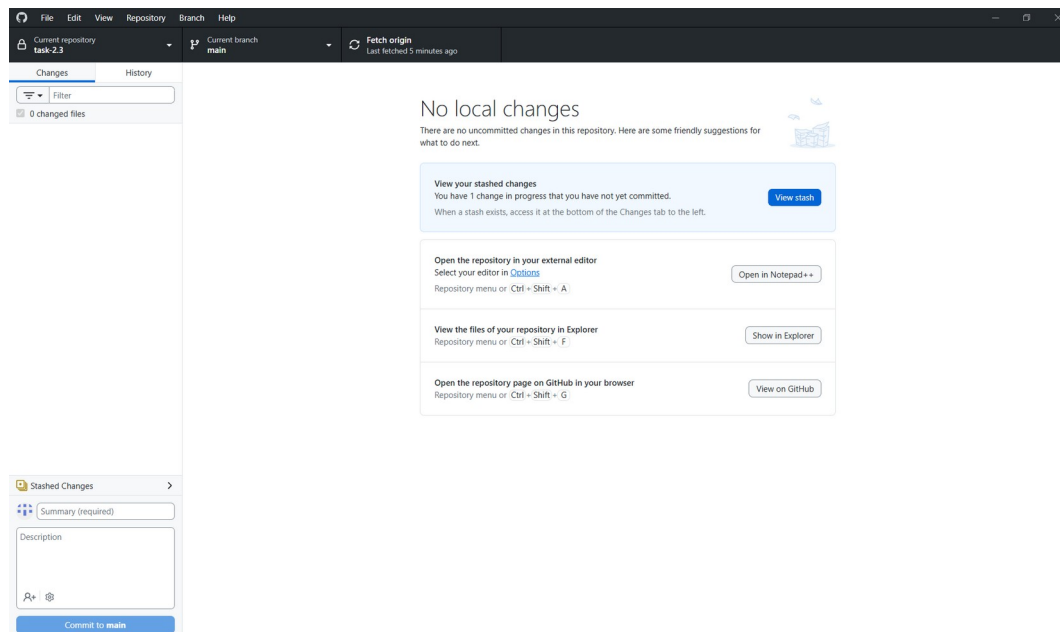
Инструменты изоляции данных и работа со скрытым буфером

Были внесены новые локальные правки, требующие временного извлечения из рабочей области.

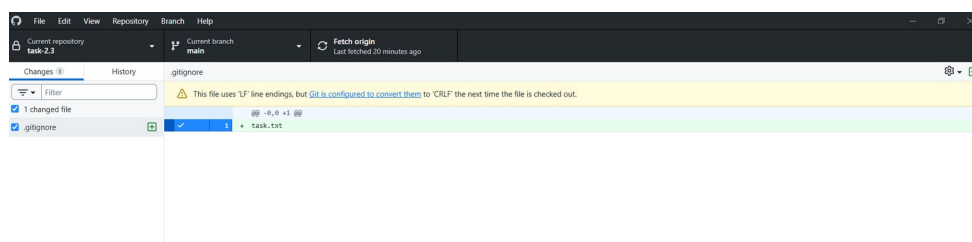


Затем был использован stash. Для этого выбрана команда Stash all changes.

Интерфейс очистил рабочую область до состояния «No local changes». При этом активировался специализированный контролирующий блок Stashed changes, уведомляющий о наличии одной сохраненной незавершенной сессии.

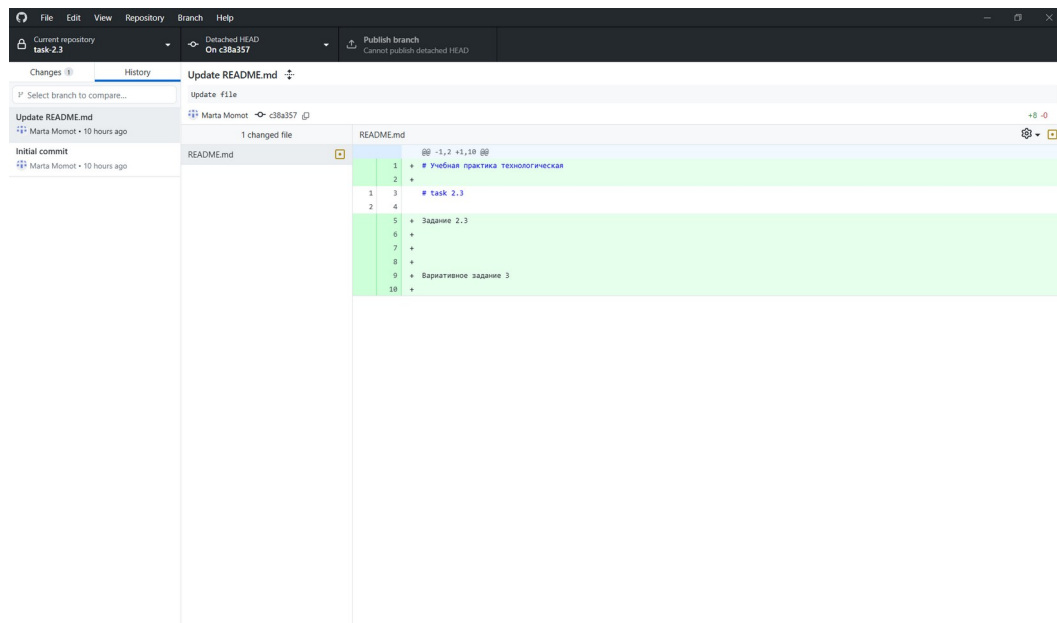


В репозитории был создан новый файл `task.txt`. Путем вызова контекстного меню файла была применена команда добавления в игнорирование. Программа автоматически создала служебный файл `.gitignore`.



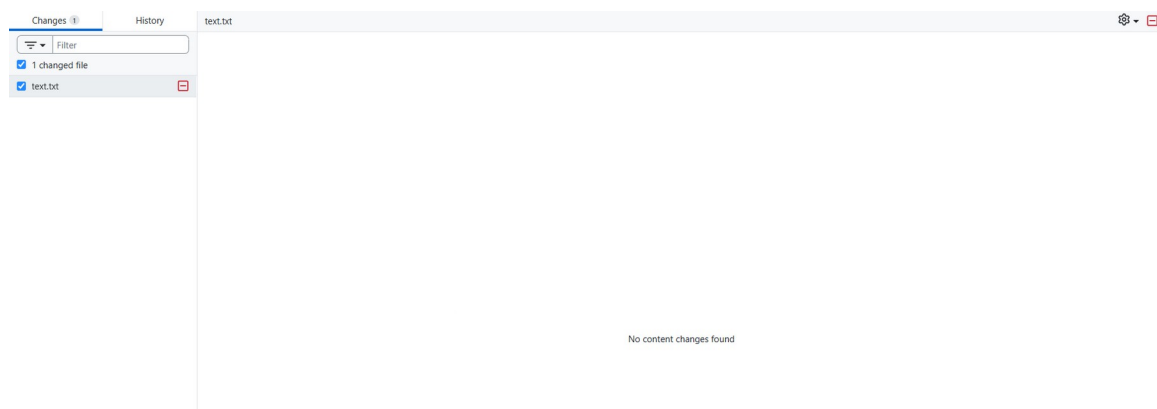
На вкладке `History` был выбран один из ранних коммитов и вызвана команда `Checkout commit`.

Графический клиент перевел репозиторий в состояние `Detached HEAD`. Файлы на жестком диске компьютера были автоматически приведены к состоянию на момент выбранного коммита, что обеспечило безопасную ревизию старого кода.



Синхронизация удаления объектов из репозитория

Из локальной рабочей директории был физически удален отслеживаемый файл text.txt. GUI-клиент идентифицировал удаление объекта. На вкладке Changes появился красный знак минус. Были заполнены обязательные поля фиксации, затем нажата кнопка Commit 1 file to main для последующей отправки на удаленный сервер командой Push origin.



В ходе лабораторной работы были изучены функции и защитные механизмы графического клиента GitHub Desktop при работе с Git. На практике был выполнен базовый цикл контроля версий, включающий локальную инициализацию репозитория, автоматическое добавление правил в .gitignore и отправку коммита удаления файлов на удаленный сервер. В процессе работы были протестированы операции ветвления, частичного строчного коммита, переноса незафиксированных изменений при переключении веток, последующего слияния веток, а также

ручного скрывания черновиков во временный буфер stash с возможностью их последующего восстановления или удаления. Тестирование защитных алгоритмов программы подтвердило блокировку опасного сброса прогресса в окне «Undo commit», успешный откат изменений через компенсирующий коммит revert, безопасную ревизию старого кода в состоянии Detached HEAD, а также сетевую синхронизацию локальной копии с сервером с помощью команд Fetch и Pull. Таким образом, GitHub Desktop заменяет команды терминала Git, защищает файлы репозитория от случайной потери данных пользователем.